```
import http.server
import socketserver
import os
import json
import socket
import mimetypes
from urllib.parse import unquote, parse_qs
from pathlib import Path
import re

# Get the folder where this script is located
SHARE_FOLDER = os.path.dirname(os.path.abspath(__file__))
PORT = 8000

def get_local_ip():
    """Get the local machine IP address"""
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
        s.close()
        return ip
    except Exception:
        return "127.0.0.1"

class CustomHTTPRequestHandler(http.server.SimpleHTTPRequestHandler):
    def do_GET(self):
        # API endpoint to list files
        if self.path.startswith("/api/list"):
            folder_path = unquote(self.path.replace("/api/list", ""))
            if not folder_path or folder_path == "/":
                folder_path = SHARE_FOLDER
            else:
                folder_path = os.path.join(SHARE_FOLDER, folder_path.lstrip("/"))

            # Security check
            if not os.path.abspath(folder_path).startswith(SHARE_FOLDER):
                self.send_error(403)
                return

            if not os.path.isdir(folder_path):
                self.send_error(404)
                return

            items = []
            try:
                for item in sorted(os.listdir(folder_path)):
                    item_path = os.path.join(folder_path, item)
                    is_dir = os.path.isdir(item_path)
                    size = 0 if is_dir else os.path.getsize(item_path)

                    items.append({
                        "name": item,
                        "isDir": is_dir,
                        "size": size,
                        "path": f"/{os.path.relpath(item_path, SHARE_FOLDER).replace(os.sep, '/')}"
                    })
            except PermissionError:
                pass

            self.send_response(200)
            self.send_header("Content-type", "application/json")
```

```
            self.end_headers()
            self.wfile.write(json.dumps(items).encode())
            return

        # Serve index.html for root
        if self.path == "/" or self.path == "":
            self.path = "/index.html"

        # Serve files
        if self.path == "/index.html":
            self.send_response(200)
            self.send_header("Content-type", "text/html")
            self.end_headers()
            self.wfile.write(get_html().encode())
            return

        # Download file
        file_path = os.path.join(SHARE_FOLDER, self.path.lstrip("/"))
        file_path = os.path.abspath(file_path)

        if not file_path.startswith(SHARE_FOLDER):
            self.send_error(403)
            return

        if os.path.isfile(file_path):
            self.send_response(200)
            mime_type, _ = mimetypes.guess_type(file_path)
            self.send_header("Content-type", mime_type or "application/octet-stream")
            self.send_header("Content-Disposition", f"attachment; filename={os.path.basename(file_path)}")
            self.end_headers()
            with open(file_path, "rb") as f:
                self.wfile.write(f.read())
            return

        self.send_error(404)

    def do_POST(self):
        """Handle file uploads"""
        if self.path.startswith("/api/upload"):
            # Parse the upload path
            upload_path = unquote(self.path.replace("/api/upload", ""))
            if not upload_path or upload_path == "/":
                target_folder = SHARE_FOLDER
            else:
                target_folder = os.path.join(SHARE_FOLDER, upload_path.lstrip("/"))

            # Security check
            if not os.path.abspath(target_folder).startswith(SHARE_FOLDER):
                self.send_error(403)
                return

            if not os.path.isdir(target_folder):
                self.send_error(404)
                return

            # Get content length
            content_length = int(self.headers.get('Content-Length', 0))
            if content_length == 0:
                self.send_error(400)
                return

            # Parse multipart form data
            content_type = self.headers.get('Content-Type', '')
            boundary_match = re.search(r'boundary=([^\s;]+)', content_type)
```

```
if not boundary_match:
    self.send_error(400)
    return

boundary = boundary_match.group(1).encode()
body = self.rfile.read(content_length)

try:
    parts = body.split(b'--' + boundary)
    uploaded_files = []

    for part in parts[1:-1]:
        if b'Content-Disposition' not in part:
            continue

        # Parse headers
        header_end = part.find(b'\r\n\r\n')
        if header_end == -1:
            header_end = part.find(b'\n\n')
            delimiter = b'\n'
        else:
            delimiter = b'\r\n'

        headers = part[:header_end].decode('utf-8', errors='ignore')
        file_content = part[header_end + 4:]

        # Remove trailing boundary markers
        if file_content.endswith(b'\r\n'):
            file_content = file_content[:-2]
        elif file_content.endswith(b'\n'):
            file_content = file_content[:-1]

        # Extract filename
        filename_match = re.search(r'filename="([^"]+)"', headers)
        if not filename_match:
            continue

        filename = filename_match.group(1)
        # Security: clean filename
        filename = os.path.basename(filename)
        if not filename:
            continue

        # Save file
        file_path = os.path.join(target_folder, filename)
        with open(file_path, 'wb') as f:
            f.write(file_content)

        uploaded_files.append({
            "name": filename,
            "size": len(file_content)
        })

    # Send response
    self.send_response(200)
    self.send_header("Content-type", "application/json")
    self.end_headers()
    response = json.dumps({"success": True, "files": uploaded_files})
    self.wfile.write(response.encode())

except Exception as e:
    self.send_response(500)
    self.send_header("Content-type", "application/json")
```

```
                self.end_headers()
                response = json.dumps({"success": False, "error": str(e)})
                self.wfile.write(response.encode())

            return

        self.send_error(404)

    def log_message(self, format, *args):
        pass  # Suppress log messages

def get_html():
    return '''<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>File Share</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
            padding: 20px;
        }

        .container {
            background: white;
            border-radius: 12px;
            box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
            width: 100%;
            max-width: 900px;
            overflow: hidden;
        }

        .header {
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            padding: 30px;
            text-align: center;
        }

        .header h1 {
            font-size: 28px;
            margin-bottom: 8px;
        }

        .breadcrumb {
            background: #f5f5f5;
            padding: 12px 20px;
            font-size: 14px;
            color: #666;
            border-bottom: 1px solid #e0e0e0;
            display: flex;
            align-items: center;
```

```
    gap: 8px;
    flex-wrap: wrap;
}

.breadcrumb a {
    color: #667eea;
    text-decoration: none;
    cursor: pointer;
    padding: 4px 8px;
    border-radius: 4px;
    transition: background 0.2s;
}

.breadcrumb a:hover {
    background: #e8e8ff;
}

.breadcrumb span {
    color: #999;
}

.content {
    padding: 20px;
    min-height: 300px;
}

.upload-section {
    margin-bottom: 30px;
}

.upload-area {
    border: 2px dashed #667eea;
    border-radius: 8px;
    padding: 30px;
    text-align: center;
    cursor: pointer;
    transition: all 0.3s;
    background: #f8f9ff;
}

.upload-area:hover {
    background: #f0f0ff;
    border-color: #764ba2;
}

.upload-area.dragover {
    background: #e8e8ff;
    border-color: #764ba2;
    box-shadow: 0 4px 12px rgba(102, 126, 234, 0.3);
}

.upload-icon {
    font-size: 48px;
    margin-bottom: 15px;
}

.upload-text {
    font-size: 16px;
    color: #333;
    margin-bottom: 8px;
    font-weight: 500;
}

.upload-subtext {
```

```
    font-size: 13px;
    color: #999;
}

#fileInput {
    display: none;
}

.upload-progress {
    display: none;
    margin-top: 15px;
}

.progress-bar {
    width: 100%;
    height: 8px;
    background: #e0e0e0;
    border-radius: 4px;
    overflow: hidden;
}

.progress-fill {
    height: 100%;
    background: linear-gradient(90deg, #667eea 0%, #764ba2 100%);
    width: 0%;
    transition: width 0.3s;
}

.upload-status {
    font-size: 13px;
    color: #667eea;
    margin-top: 8px;
    font-weight: 500;
}

.file-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
    gap: 15px;
    margin-bottom: 20px;
}

.file-item {
    background: #f9f9f9;
    border: 1px solid #e0e0e0;
    border-radius: 8px;
    padding: 15px;
    text-align: center;
    cursor: pointer;
    transition: all 0.3s;
    text-decoration: none;
    color: #333;
    position: relative;
}

.file-item:hover {
    background: #f0f0ff;
    border-color: #667eea;
    transform: translateY(-2px);
    box-shadow: 0 4px 12px rgba(102, 126, 234, 0.2);
}

.file-icon {
    font-size: 36px;
```

```
    margin-bottom: 10px;
}

.file-name {
    font-size: 13px;
    font-weight: 500;
    word-break: break-word;
    overflow: hidden;
    text-overflow: ellipsis;
    display: -webkit-box;
    -webkit-line-clamp: 2;
    -webkit-box-orient: vertical;
}

.file-size {
    font-size: 11px;
    color: #999;
    margin-top: 8px;
}

.empty-state {
    text-align: center;
    padding: 60px 20px;
    color: #999;
}

.empty-state-icon {
    font-size: 64px;
    margin-bottom: 20px;
    opacity: 0.5;
}

.loader {
    text-align: center;
    padding: 40px 20px;
    color: #667eea;
}

.spinner {
    border: 4px solid #f0f0f0;
    border-top: 4px solid #667eea;
    border-radius: 50%;
    width: 40px;
    height: 40px;
    animation: spin 0.8s linear infinite;
    margin: 0 auto 15px;
}

@keyframes spin {
    0% { transform: rotate(0deg); }
    100% { transform: rotate(360deg); }
}

.footer {
    background: #f5f5f5;
    padding: 15px 20px;
    text-align: center;
    font-size: 12px;
    color: #999;
    border-top: 1px solid #e0e0e0;
}

.alert {
    padding: 12px 16px;
```

```
            border-radius: 6px;
            margin-bottom: 15px;
            font-size: 14px;
        }

        .alert-success {
            background: #d4edda;
            border: 1px solid #c3e6cb;
            color: #155724;
        }

        .alert-error {
            background: #f8d7da;
            border: 1px solid #f5c6cb;
            color: #721c24;
        }

        @media (max-width: 600px) {
            .file-grid {
                grid-template-columns: repeat(auto-fill, minmax(120px, 1fr));
                gap: 10px;
            }

            .header h1 {
                font-size: 20px;
            }

            .upload-area {
                padding: 20px;
            }

            .upload-icon {
                font-size: 36px;
            }
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>📁 File Share</h1>
        </div>

        <div class="breadcrumb" id="breadcrumb"></div>

        <div class="content" id="content">
            <div class="upload-section">
                <div class="upload-area" id="uploadArea">
                    <div class="upload-icon">⬆️</div>
                    <div class="upload-text">Drag files here or click to select</div>
                    <div class="upload-subtext">Multiple files supported</div>
                </div>
                <input type="file" id="fileInput" multiple>
                <div class="upload-progress" id="uploadProgress">
                    <div class="progress-bar">
                        <div class="progress-fill" id="progressFill"></div>
                    </div>
                    <div class="upload-status" id="uploadStatus">Uploading...</div>
                </div>
            </div>

            <div id="alerts"></div>

            <div class="loader" id="loader">
```

```
            <div class="spinner"></div>
            <p>Loading...</p>
        </div>

        <div class="file-grid" id="fileGrid"></div>
    </div>

    <div class="footer">
        Sharing folder contents
    </div>
</div>

<script>
    let currentPath = "/";

    const icons = {
        folder: "📁",
        image: "🖼️",
        video: "🎬",
        audio: "🎵",
        pdf: "📄",
        zip: "📦",
        code: "💻",
        text: "📝",
        file: "📄"
    };

    const uploadArea = document.getElementById("uploadArea");
    const fileInput = document.getElementById("fileInput");
    const uploadProgress = document.getElementById("uploadProgress");
    const progressFill = document.getElementById("progressFill");
    const uploadStatus = document.getElementById("uploadStatus");
    const alerts = document.getElementById("alerts");

    uploadArea.addEventListener("click", () => fileInput.click());
    uploadArea.addEventListener("dragover", (e) => {
        e.preventDefault();
        uploadArea.classList.add("dragover");
    });
    uploadArea.addEventListener("dragleave", () => {
        uploadArea.classList.remove("dragover");
    });
    uploadArea.addEventListener("drop", (e) => {
        e.preventDefault();
        uploadArea.classList.remove("dragover");
        handleFiles(e.dataTransfer.files);
    });

    fileInput.addEventListener("change", (e) => {
        handleFiles(e.target.files);
    });

    function handleFiles(files) {
        if (files.length === 0) return;

        const formData = new FormData();
        for (let file of files) {
            formData.append("files", file);
        }

        uploadProgress.style.display = "block";
        progressFill.style.width = "0%";
        uploadStatus.textContent = `Uploading ${files.length} file(s)...`;
```

```
    fetch(`/api/upload${currentPath}`, {
        method: "POST",
        body: formData
    })
    .then(res => res.json())
    .then(data => {
        uploadProgress.style.display = "none";
        fileInput.value = "";

        if (data.success) {
            showAlert(`✓ Successfully uploaded ${data.files.length} file(s)!`, "success");
            loadFiles();
        } else {
            showAlert(`× Upload failed: ${data.error}`, "error");
        }
    })
    .catch(err => {
        uploadProgress.style.display = "none";
        showAlert(`× Upload error: ${err.message}`, "error");
    });
}

function showAlert(message, type) {
    const alertDiv = document.createElement("div");
    alertDiv.className = `alert alert-${type}`;
    alertDiv.textContent = message;
    alerts.appendChild(alertDiv);

    setTimeout(() => {
        alertDiv.remove();
    }, 5000);
}

function getFileIcon(name, isDir) {
    if (isDir) return icons.folder;

    const ext = name.split(".").pop().toLowerCase();

    if (["jpg", "jpeg", "png", "gif", "svg", "webp"].includes(ext)) return icons.image;
    if (["mp4", "avi", "mkv", "mov"].includes(ext)) return icons.video;
    if (["mp3", "wav", "flac", "m4a"].includes(ext)) return icons.audio;
    if (ext === "pdf") return icons.pdf;
    if (["zip", "rar", "7z", "tar"].includes(ext)) return icons.zip;
    if (["js", "py", "java", "cpp", "html", "css"].includes(ext)) return icons.code;
    if (["txt", "md", "doc"].includes(ext)) return icons.text;

    return icons.file;
}

function formatFileSize(bytes) {
    if (bytes === 0) return "-";
    const k = 1024;
    const sizes = ["B", "KB", "MB", "GB"];
    const i = Math.floor(Math.log(bytes) / Math.log(k));
    return Math.round((bytes / Math.pow(k, i)) * 100) / 100 + " " + sizes[i];
}

function updateBreadcrumb() {
    const parts = currentPath.split("/").filter(p => p);
    let html = `<a onclick="navigate('/')">Home</a>`;

    let path = "";
    for (let part of parts) {
        path += "/" + part;
```

```
            html += `<span>/</span><a onclick="navigate('${path}')">${part}</a>`;
        }

        document.getElementById("breadcrumb").innerHTML = html;
    }

    function navigate(path) {
        currentPath = path;
        loadFiles();
    }

    function loadFiles() {
        updateBreadcrumb();

        const loader = document.getElementById("loader");
        const fileGrid = document.getElementById("fileGrid");
        loader.style.display = "block";
        fileGrid.innerHTML = "";

        fetch(`/api/list${currentPath}`)
            .then(res => res.json())
            .then(data => {
                loader.style.display = "none";

                if (data.length === 0) {
                    fileGrid.innerHTML = `<div class="empty-state" style="grid-column: 1/-1;">
                        <div class="empty-state-icon">📭</div>
                        <p>This folder is empty</p>
                    </div>`;
                    return;
                }

                for (let item of data) {
                    const icon = getFileIcon(item.name, item.isDir);
                    const size = formatFileSize(item.size);

                    if (item.isDir) {
                        const div = document.createElement("div");
                        div.className = "file-item";
                        div.onclick = () => navigate(item.path);
                        div.innerHTML = `
                            <div class="file-icon">${icon}</div>
                            <div class="file-name">${item.name}</div>
                        `;
                        fileGrid.appendChild(div);
                    } else {
                        const a = document.createElement("a");
                        a.className = "file-item";
                        a.href = item.path;
                        a.download = true;
                        a.innerHTML = `
                            <div class="file-icon">${icon}</div>
                            <div class="file-name">${item.name}</div>
                            <div class="file-size">${size}</div>
                        `;
                        fileGrid.appendChild(a);
                    }
                }
            })
            .catch(err => {
                loader.style.display = "none";
                fileGrid.innerHTML = `<div class="empty-state" style="grid-column: 1/-1;">
                    <div class="empty-state-icon">❌</div>
                    <p>Error loading files</p>
```

```
                    </div>`;
                });
        }

        loadFiles();
    </script>
</body>
</html>'''

os.chdir(SHARE_FOLDER)
local_ip = get_local_ip()

with socketserver.TCPServer(("", PORT), CustomHTTPRequestHandler) as httpd:
    print("\n" + "="*50)
    print("📁 FILE SHARE SERVER STARTED")
    print("="*50)
    print(f"📂 Sharing folder: {SHARE_FOLDER}")
    print(f"🌐 Local access:   http://localhost:{PORT}")
    print(f"🌐 Network access: http://{local_ip}:{PORT}")
    print("="*50)
    print("Press Ctrl+C to stop\n")
    httpd.serve_forever()
```